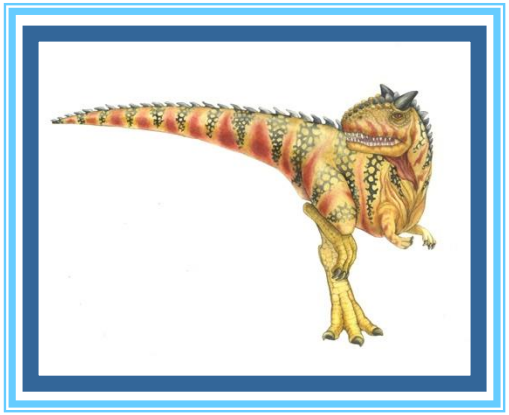
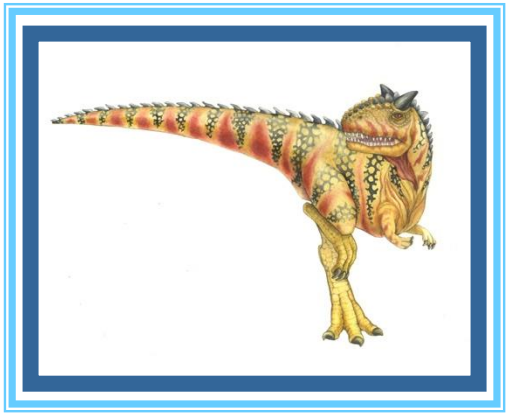
These slides have been modified from the original



A huge thanks to the authors of Operating System Concepts for providing this resource

These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides

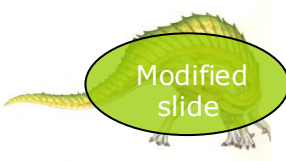
Modified slide

What is an Operating System?

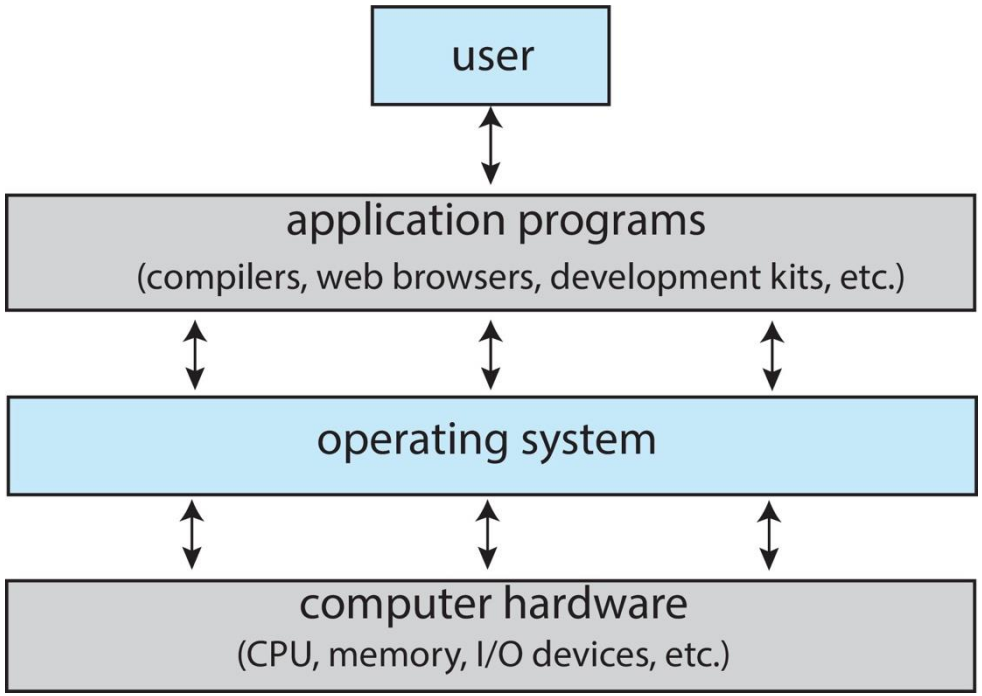
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

Computer System Structure

- Computer system can be divided into **four components**:
  - **Hardware** – provides basic computing resources
    - CPU, memory, I/O devices, ...
  - **Operating system**
    - Controls and coordinates use of hardware among various applications and users
  - **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games, ...
  - **Users**
    - People, machines, other computers



Abstract View of Components of Computer



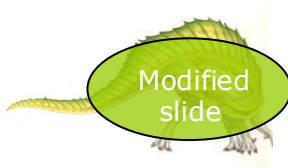
Operating System Definition

- No universally accepted definition

"*Everything a vendor ships when you order an operating system*" is a good approximation

  - But varies wildly

- The one program running at all times on the computer is the **kernel,** part of the operating system
- Everything else is either
  - A ***system program*** (ships with the operating system, but not part of the kernel) , or
  - An ***application program***, all programs not associated with the operating system



Some major/common operating systems

- Microsoft Windows (3.1 / 95 / 98 / 2000 / NT / Me / XP / 10 / 11 / Server)
- POSIX Compliant (or mostly compliant) operating systems
  - **Linux**: MX Linux, EndeavourOS, Manjaro, Mint, Pop!_OS, Ubuntu, Debian, Garuda, ... The list goes on – see distrowatch.com
  - **BSDs**: FreeBSD, NetBSD, Darwin, DragonFly, ...
- Some are proprietary (Windows), some are not (Linux, BSD), some are for embedded devices (QNX, VxWorks, WinCE...). RTOS.
- Get a sense of the number of these (and the classifications used): https://en.wikipedia.org/wiki/List_of_operating_systems

Different supported platforms

- For example:
  - Intel / AMD x86 / x86_64

  - ARM in Mac M1/M2/M3/M4

  - ARM Cortex-A72

And they have different goals

- For example:
  - OpenBSD
  - ... The focus is on security, correctness, ...

And they have different goals

- For example:
  - NetBSD
  - ... The focus is portability

Platforms Supported by NetBSD

Platforms (a.k.a. 'ports')

- Ports
- The tier system

Tiers

- Tier I: Focus — support is part of NetBSD's strategy
- Tier II: Organic — evolving at its own pace
- Tier III: Life Support — severely incapacitated or broken

CPU architectures

- Ports by CPU architecture

Platforms (a.k.a. 'ports')

Ports

NetBSD calls a supported architecture a *port*. Most ports run on generic hardware and emulators, although some commercial hardware also exists. The NetBSD Ports History page details the inclusion date for each port.

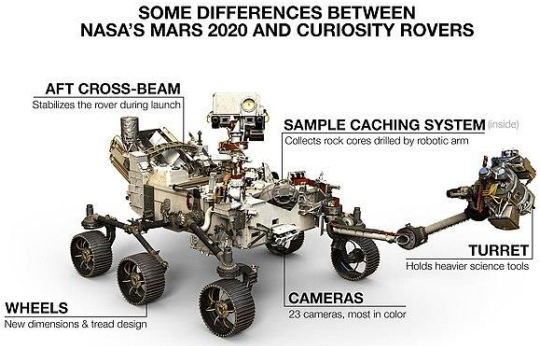
The tier system

Ports are classified into three tiers based on the current importance of the architecture and the level of community activity. Summarizing, the tiers can be viewed to represent ports that NetBSD will support, ports that NetBSD does its best to support, and ports which may be desupported soon. The tier for each port may change over time and is decided by <core@NetBSD.org> based on input from users and developers.

More exact guidelines for tiers along with the port taxonomy is presented below.

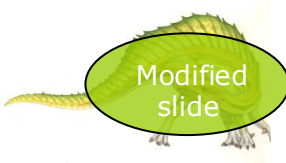
Tiers

Embedded OS devices



Real-Time Embedded Systems

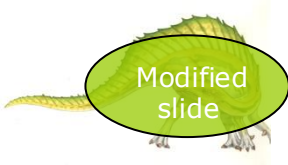
- Real-time embedded systems most prevalent form of computers
  - Vary considerably, **special purpose**, limited purpose OS, **real-time OS**
- Many other special computing environments as well
  - Some have OSes, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
  - Processing ***must*** be done within constraint
  - Correct operation only if constraints met
  - "***a late answer is a wrong answer***"



Free and Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source** and **proprietary**
- Counter to the **copy protection** and **Digital Rights Management** (**DRM**) movement
- Started by **Free Software Foundation** (**FSF**), which has "copyleft" **GNU Public License** (**GPL**)
  - **https://www.gnu.org/philosophy/open-source-misses-the-point.en.html**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more

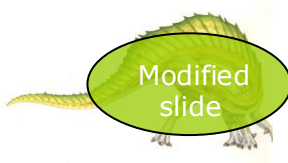
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - http://www.virtualbox.com)
  - Use to run guest operating systems for exploration



Computer-System Architecture

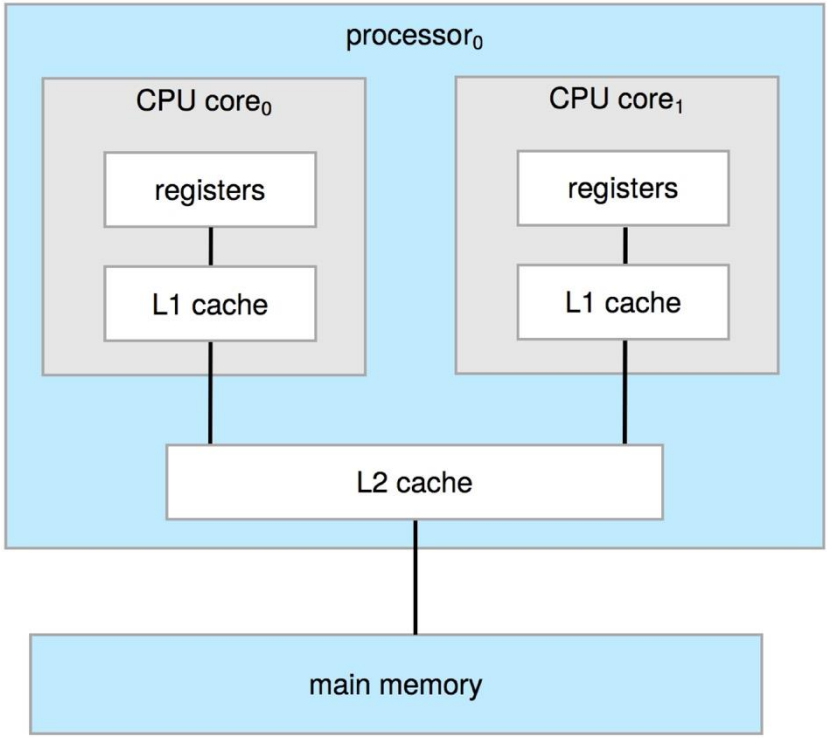
- Historically, most systems used a single general-purpose processor
  - Aside from special-purpose processors e.g. GPU/TPU/...

- **Multiprocessors** systems have grown in use and importance
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability** – graceful degradation or fault tolerance
  - Two types:
    1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
    2. **Symmetric Multiprocessing** – each processor performs all tasks (abbreviated as SMP)

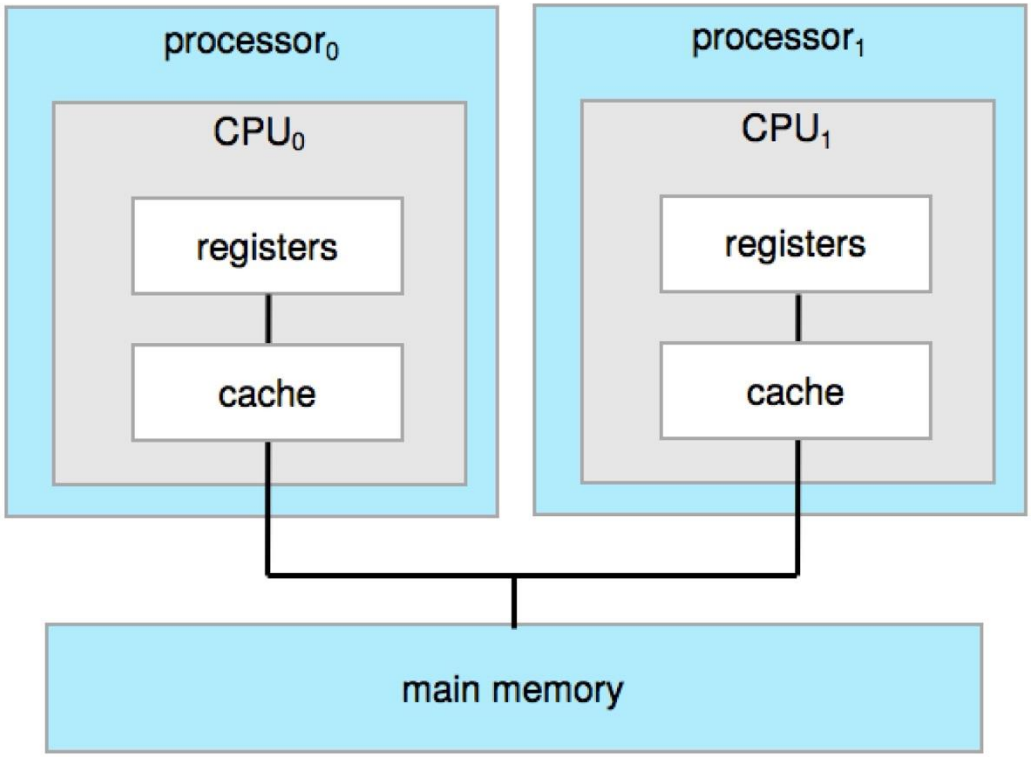


Symmetric Multiprocessing Architecture

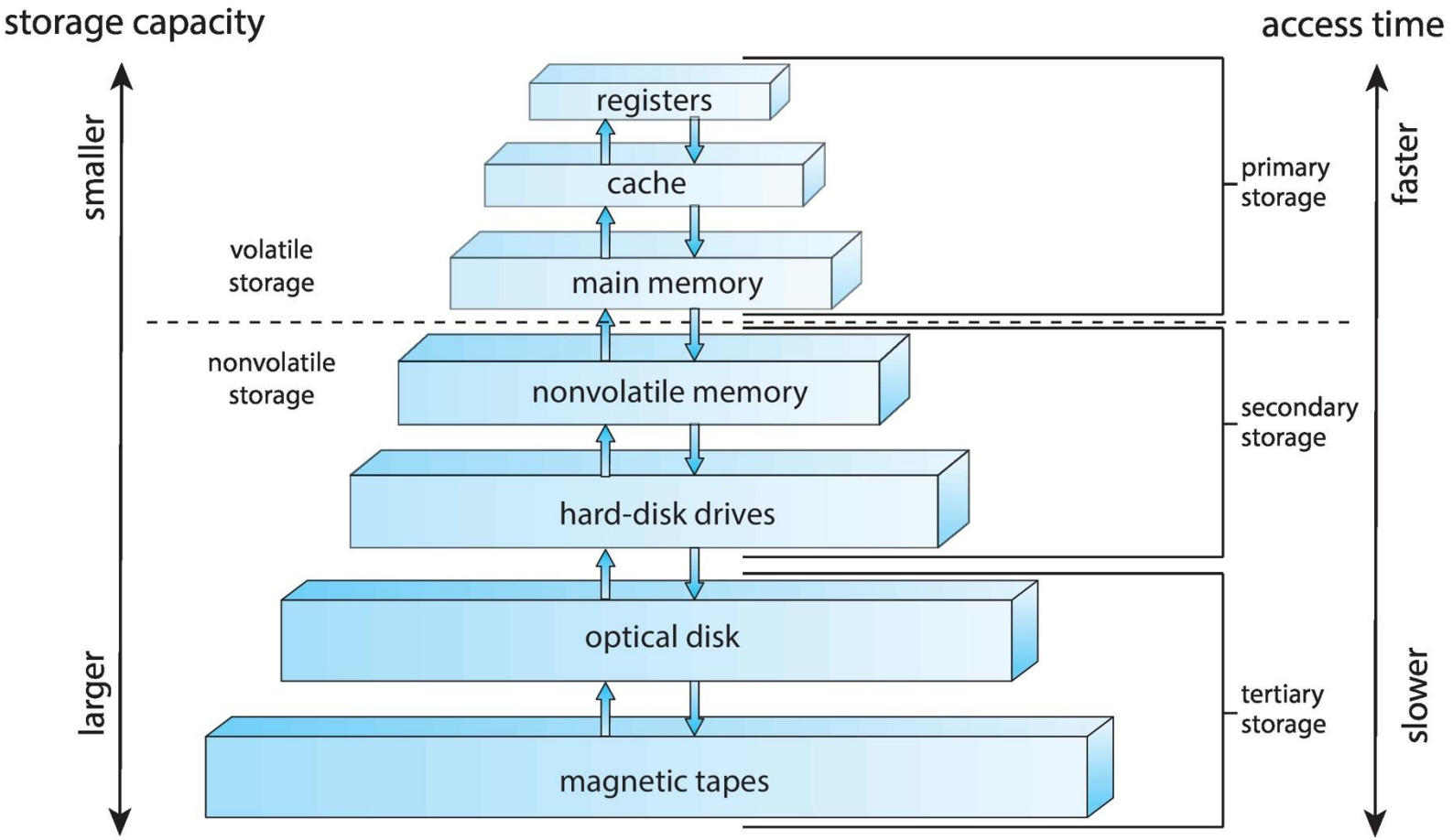
- Multi-chip and **multicore**
- Systems containing all chips
  - Chassis containing multiple separate systems



Symmetric Multiprocessing Architecture



Storage-Device Hierarchy

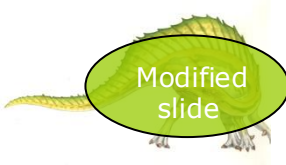


580 terrabyte magnetic tape? for the curious reader see https://techxplore.com/news/2020-12-fujifilm-ibm-unveil-terabyte-magnetic.html

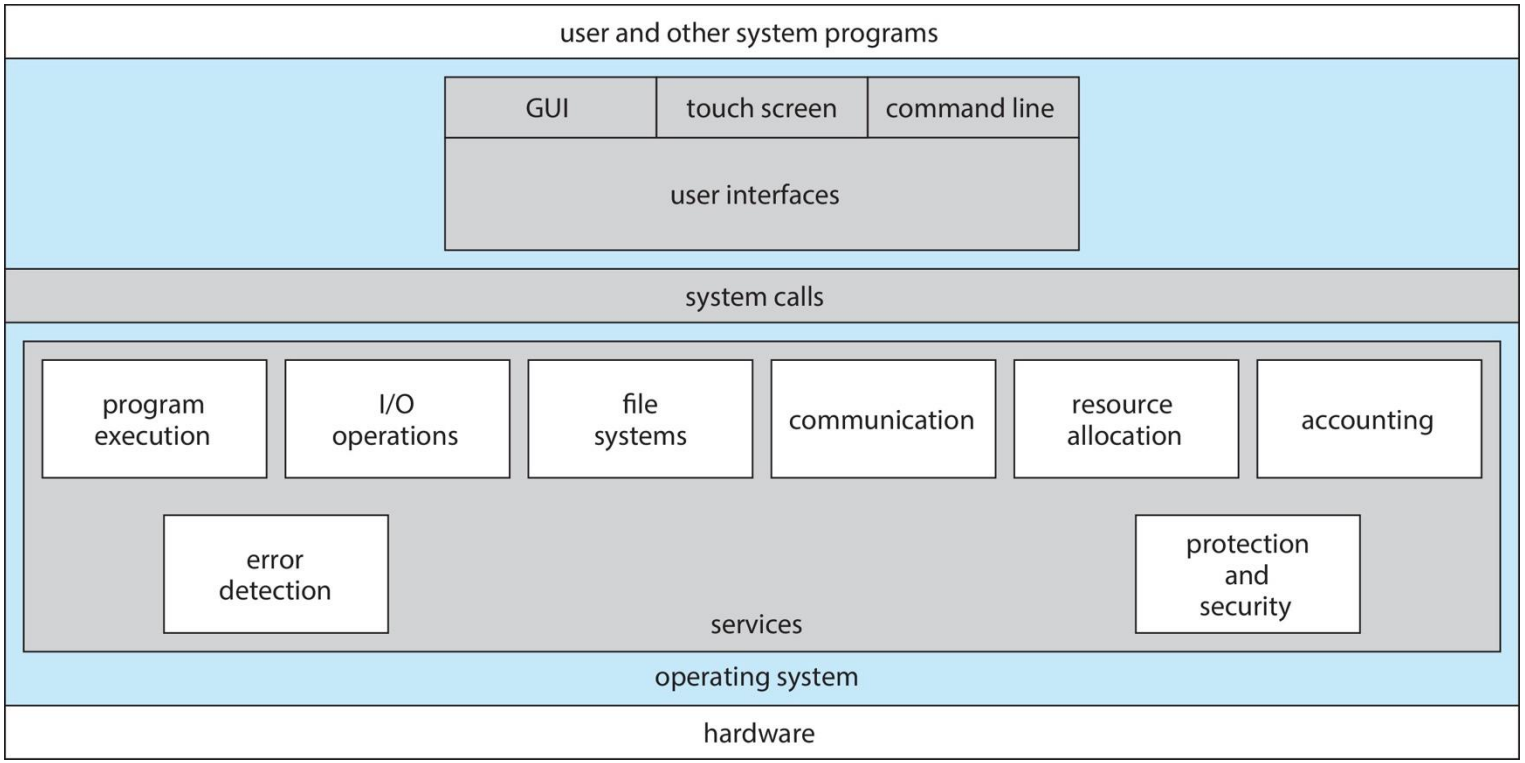
Characteristics of Various Types of Storage

Level	Name	Typical size	Technology	Access time (ns)	Bandwidth	Managed by
1	Registers	< 1 KB	CMOS flip-flops	~0.2–1	100+ GB/s	compiler + hardware
2	Cache (L1–L3)	KB → 100+ MB	SRAM	0.5–30	10–100 GB/s	hardware
3	Main memory	GB → TB	DRAM	60–150	10–50 GB/s	OS
4	SSD (NVMe)	0.5–8 TB	Flash	10^4–10^5	1–7 GB/s	OS
5	HDD / Tape	10 TB+	Magnetic	10^6–10^9	100 MB/s	OS

Indicative
M1 Ultra has 48 MB cache for exampe

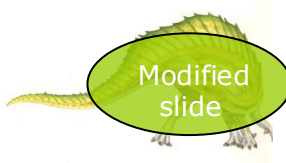


A View of Operating System Services



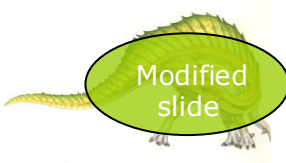
Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- Operating-system services provides functions that are helpful to the user:
  - **User interface** - Almost all operating systems have a user interface (**UI**).
    - Varies between **Command-Line** (**CLI**), **Graphics User Interface** (**GUI**), **touch-screen, ...**
  - **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations** - A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation** - Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.



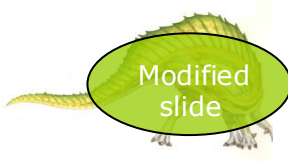
Operating System Services (Cont.)

- One set of operating-system services provides functions that are helpful to the user (Cont.):
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)
  - **Error detection** – OS needs to be constantly aware of possible errors
    - May occur in the CPU and memory hardware, in I/O devices, in user program, ...
    - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
    - Provide debugging facilities ...



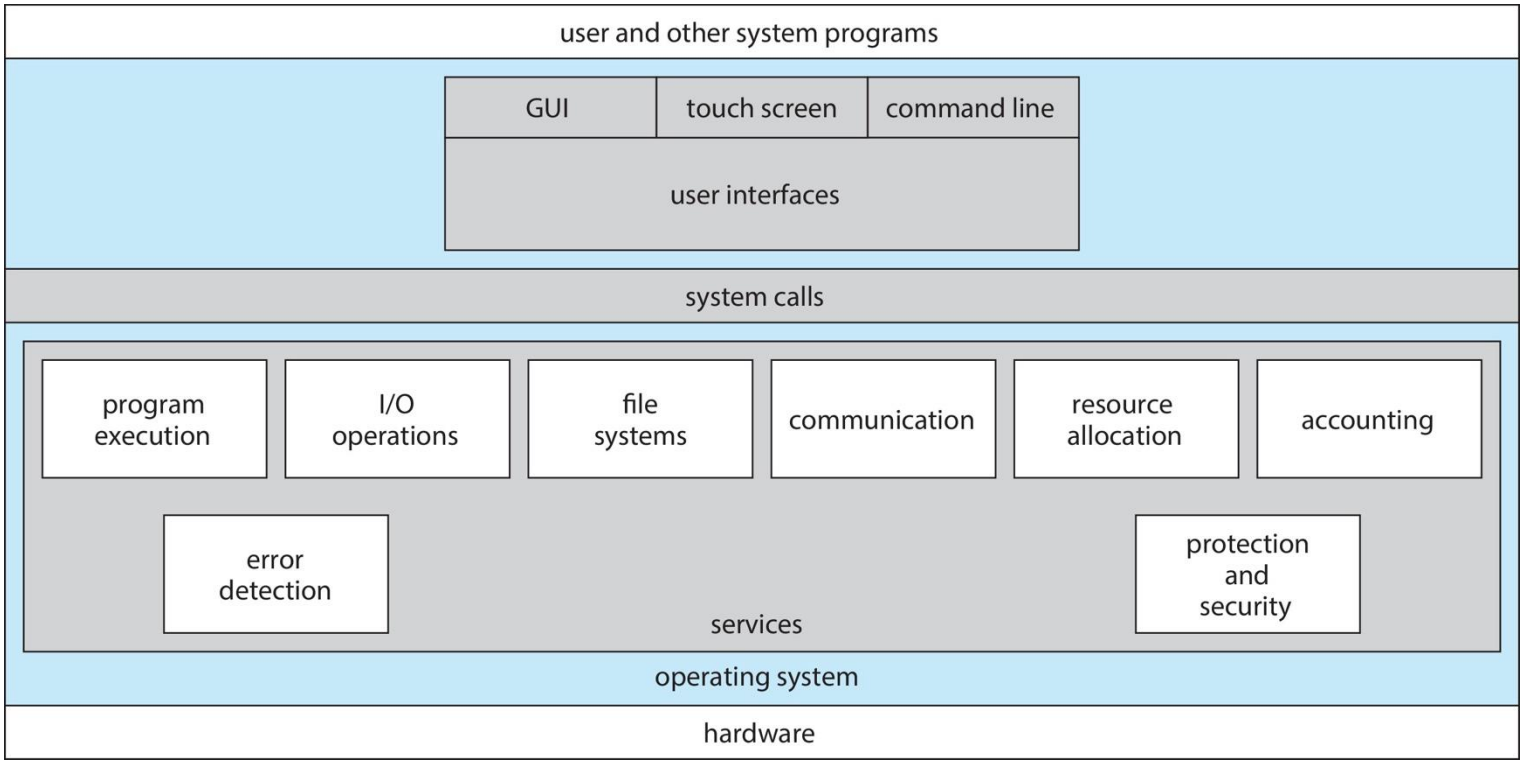
Operating System Services (Cont.)

- Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing
  - **Resource allocation -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
    - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
  - **Logging -** To keep track of which users use how much and what kinds of computer resources
  - **Protection and security -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, *processes* should not interfere with each other
    - **Protection** involves ensuring that all access to system resources is controlled
    - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



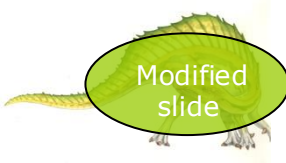
System Calls

How do programs interact with/use/control these services?



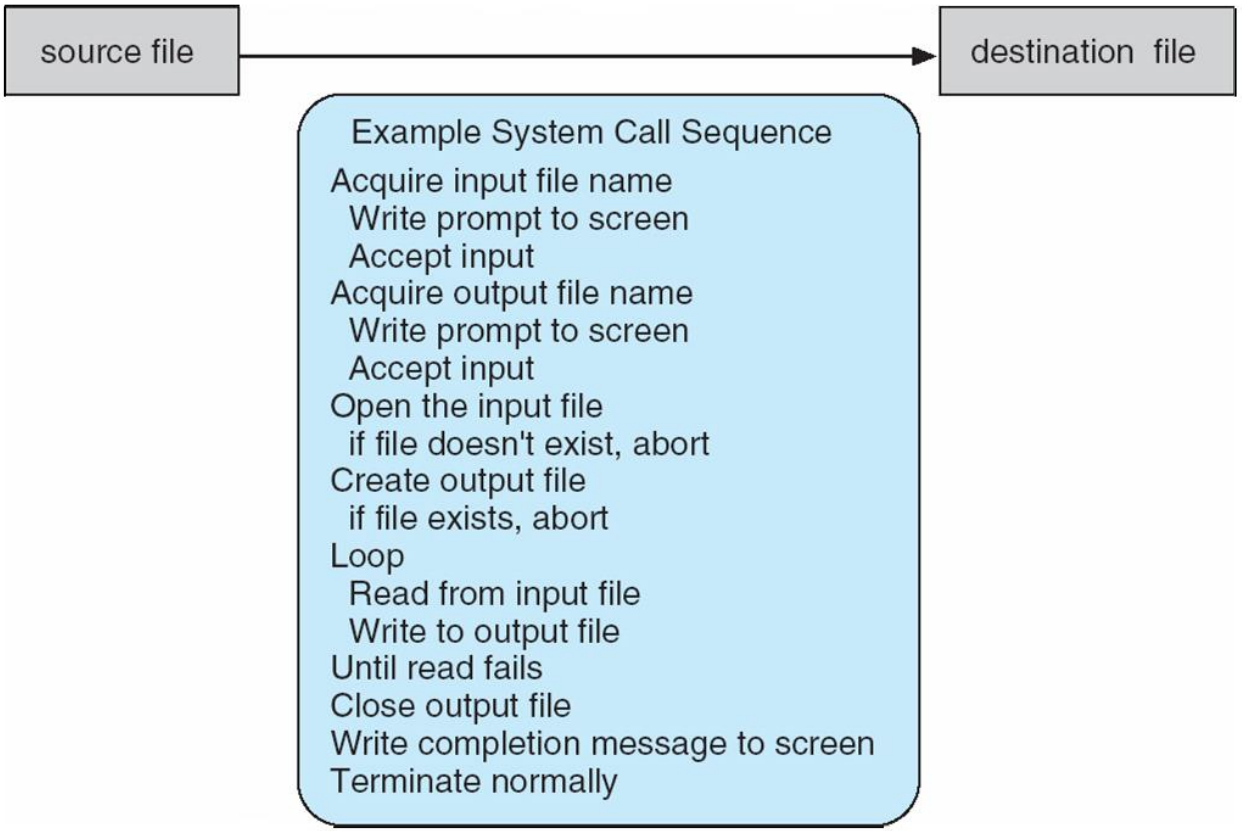
System Calls

- System calls enable software to use/interact/do things with the operating system kernel to make it do something
- In effect, it's a programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Not typically used directly .. You probably not even aware of them! Mostly accessed by programs via a high-level **Application Programming Interface** (**API**) rather than direct system call use
- Common APIs are Win API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X)



Example of System Calls

- System call sequence to copy the contents of one file to another file



Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value — function name — parameters

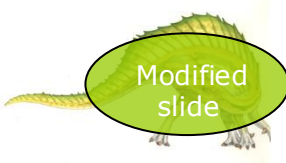
A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

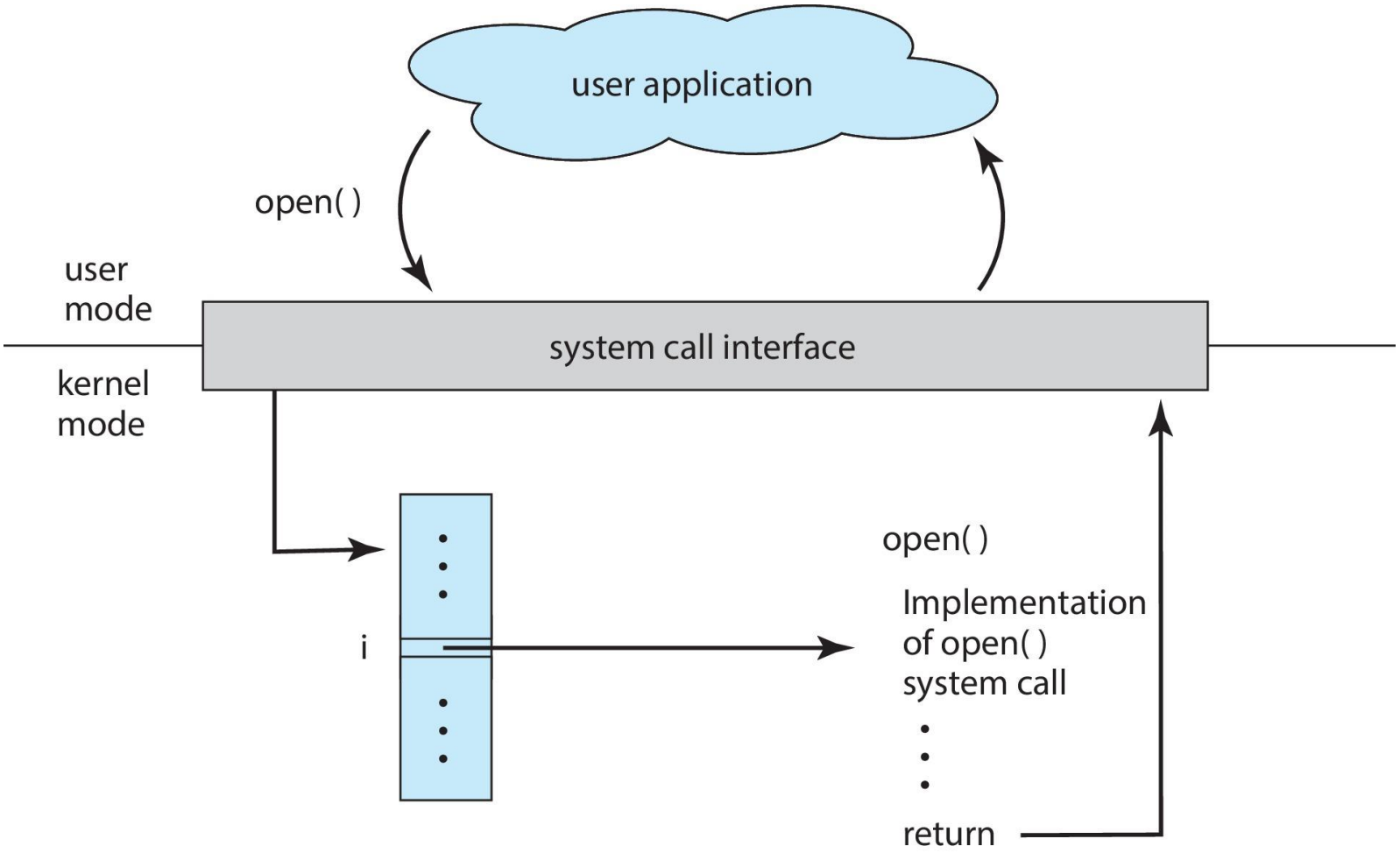
On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns –1.

System Call Implementation

- Typically, a number is associated with each system call
  - **System-call interface** maintains a table indexed according to these numbers
- The **system call interface** invokes the intended system call in OS **kernel** and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
  - Just needs to **obey API and understand what OS will do as a result call**
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)



API – System Call – OS Relationship



Types of System Calls

- Process control
  - create process, terminate process
  - end, abort
  - load, execute
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory
  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes

Types of System Calls (Cont.)

- File management
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- Device management
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices

Types of System Calls (Cont.)

- Information maintenance
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- Communications
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

Types of System Calls (Cont.)

- Protection
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

Examples of Windows and Unix System Calls

Look yourself: man syscalls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

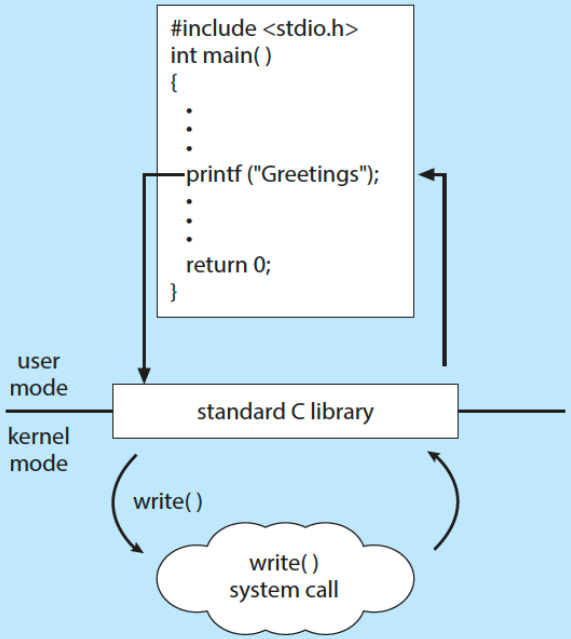
	Windows	Unix
Process control	`CreateProcess()`	`fork()`
	`ExitProcess()`	`exit()`
	`WaitForSingleObject()`	`wait()`
File management	`CreateFile()`	`open()`
	`ReadFile()`	`read()`
	`WriteFile()`	`write()`
	`CloseHandle()`	`close()`
Device management	`SetConsoleMode()`	`ioctl()`
	`ReadConsole()`	`read()`
	`WriteConsole()`	`write()`
Information maintenance	`GetCurrentProcessID()`	`getpid()`
	`SetTimer()`	`alarm()`
	`Sleep()`	`sleep()`
Communications	`CreatePipe()`	`pipe()`
	`CreateFileMapping()`	`shm_open()`
	`MapViewOfFile()`	`mmap()`
Protection	`SetFileSecurity()`	`chmod()`
	`InitializeSecurityDescriptor()`	`umask()`
	`SetSecurityDescriptorGroup()`	`chown()`

Standard C Library Example

- C program invoking printf() library call, which calls write() system call

THE STANDARD C LIBRARY

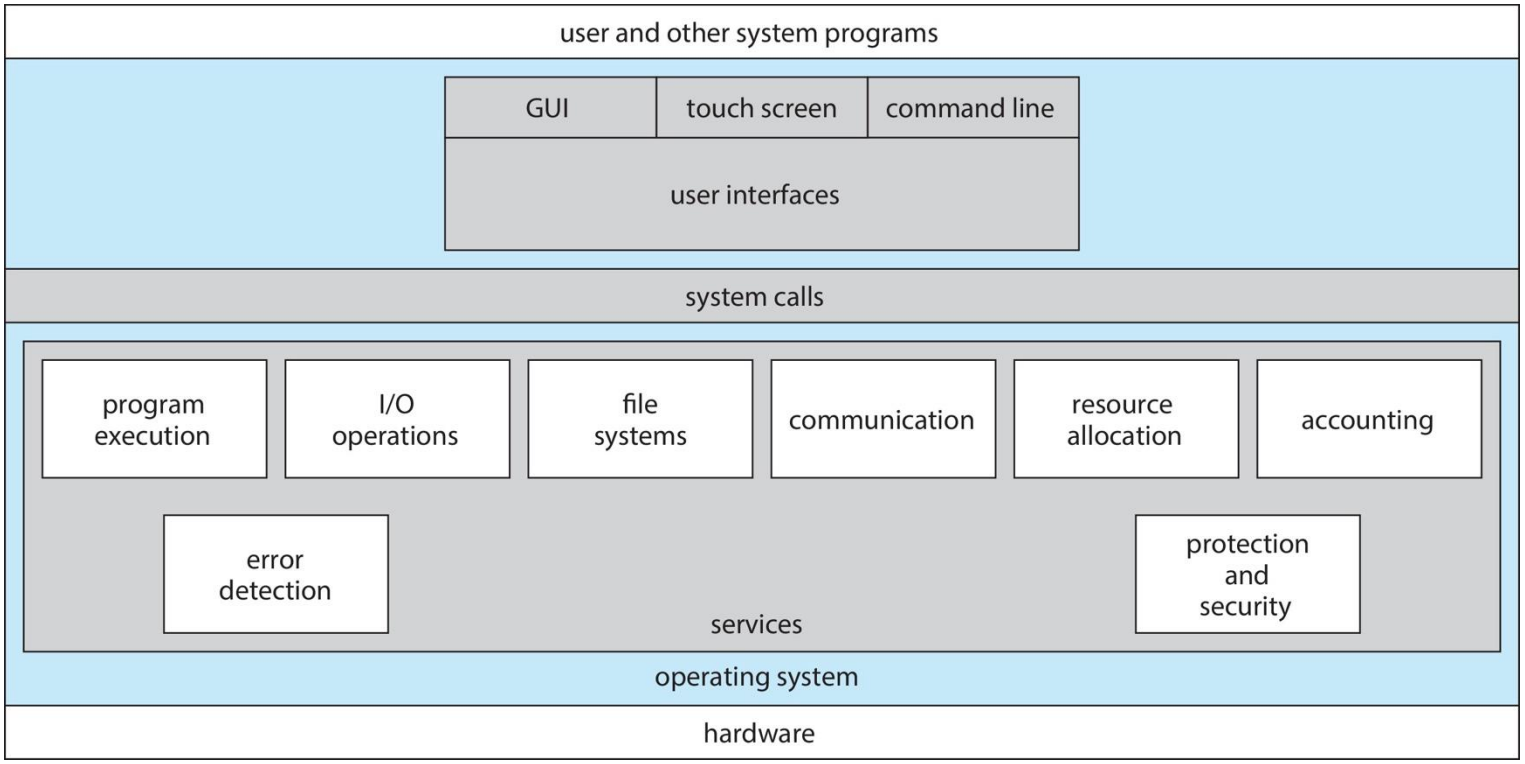
The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



Quick detour – What's a kernel?

- If we think of an operating system (as layers)
  - Application Program e.g., opening a file in a text editor
  - Libraries
  - ...
  - Libraries
    - System calls
  - Kernel e.g., Linux Kernel
  - Hardware

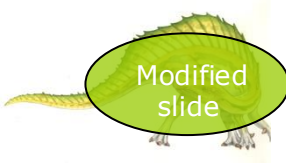
System Programs



System Services / Programs

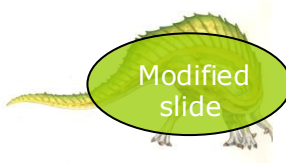
- Most users' view of the operating system **is defined by system programs, not the actual system calls**

- There are system they provide a convenient environment for program development and execution. They can be divided into:
  - File manipulation
  - Status information sometimes stored in a file
  - Programming language support
  - Program loading and execution
  - Communications
  - Background services
  - Application programs



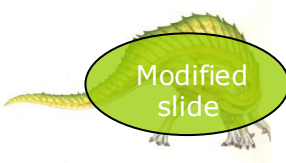
System Services / Programs

- Provide a convenient environment for program development and execution
  - Some of them are simply user interfaces to system calls; others are considerably more complex
- **File management** – Create (touch, mkdir), delete (rm), copy (cp), rename, print, dump, list (ls, find), and generally manipulate files and directories
- **Status information**
  - Some ask the system for info – date (date), time, amount of available memory (free, top), disk space (du), number of users (w)
  - Others provide detailed performance, logging, and debugging information (dmesg)
  - Typically, these programs format and print the output to the terminal or other output devices
  - Some systems implement a **registry** - used to store and retrieve configuration information (linux ... /etc/)



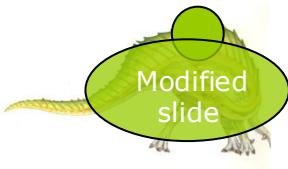
System Services / Programs

- **File modification**
  - Text editors to create and modify files (nano, vim, emacs)
  - Special commands to search contents of files or perform transformations of the text (sed, gawk, ...)
- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided (gcc, python, nasm, ...)
- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language
- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems
  - Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another (ssh, sshd, netcat, ...)



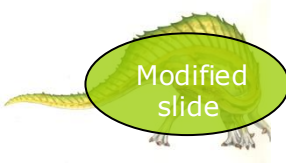
System Services / Programs

- **Background Services**
  - Launch at boot time
    - Some for system startup, then terminate
    - Some from system boot to shutdown
  - Provide facilities like disk checking, process scheduling, error logging, printing
  - Run in user context not kernel context
  - Known as **services**, **subsystems**, **daemons**
  - **Look in /etc/init.d (to get an idea)**
- **Application programs**
  - Don't pertain to system
  - Run by users
  - Not typically considered part of OS
  - Launched by command line, mouse click, finger poke



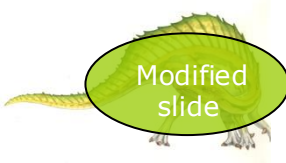
Design and Implementation

- Design and Implementation of OS is not “solvable”, but some **approaches have proven successful**
- Internal structure of different Operating Systems can vary widely
- Start the design by defining goals and specifications
- Affected by choice of **hardware**, type of **system**
- **User** goals and **System** goals
- Specifying and designing an OS is highly **creative task** of **software engineering**



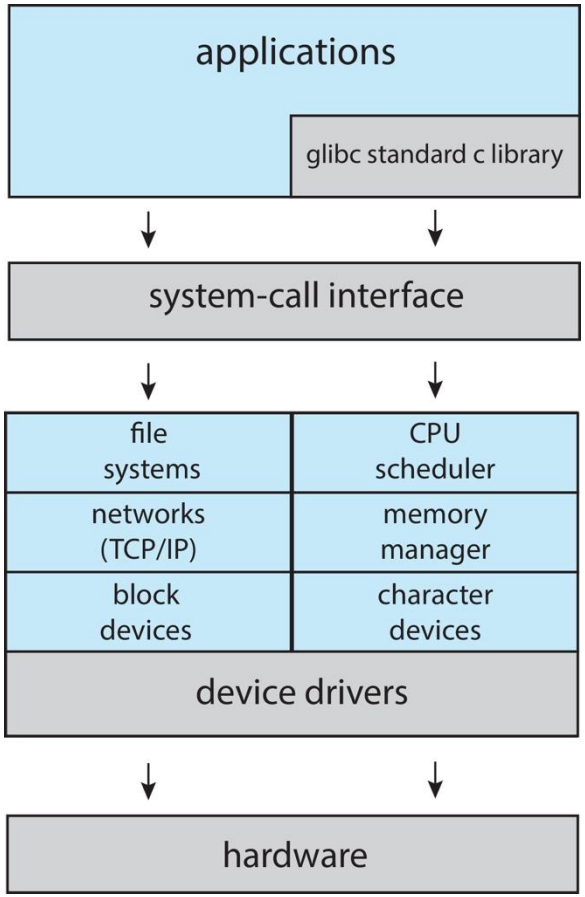
Implementation

- Much variation
  - Early OSes in assembly language
  - Then system programming languages like Algol, PL/1
  - Now C, C++
- Actually usually a mix of languages
  - Lowest levels in assembly
  - Main body in C
  - Systems programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language easier to **port** to other hardware
  - But slower
- **Emulation** can allow an OS to run on non-native hardware



Linux System Structure

Monolithic plus modular design



Building and Booting Linux

- Download Linux source code (http://www.kernel.org)
- Configure kernel via "`make menuconfig`"
- Compile the kernel using "`make`"
  - Produces `vmlinuz`, the kernel image
  - Compile kernel modules via "`make modules`"
  - Install kernel modules into `vmlinuz` via "`make modules_install`"
  - Install new kernel on the system via "`make install`"

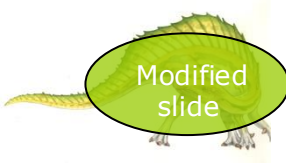
System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
  - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
  - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
  - Modern systems replace BIOS with **Unified Extensible Firmware Interface** (**UEFI**)
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode

Tracing

- Collects data for a specific event, such as steps involved in a system call invocation

- Tools include
  - strace – trace system calls invoked by a process
  - gdb – source-level debugger
  - perf – collection of Linux performance tools
  - tcpdump – collects network packets
  - ....



Linux bcc/BPF Tracing Tools